

Python Programming and Data Structures

Technical Notes

Rodrigo Kang

Table of contents

1	Introduction	1
1.1	Python as a Programming Language	3
2	Objects, Types, and Names	3
2.1	Dynamic Typing	5
2.2	Assignment and Aliasing	6
2.3	Identity and Equality	8
2.4	Mutability and Immutability	9
2.5	Copying Objects	12
3	Expressions and Operators	15
3.1	Boolean Expressions and Logical Operators	16
3.1.1	Truth Values	18
3.1.2	Short-Circuit Evaluation	19
4	Control Flow	20
4.1	Conditional Execution	21
4.2	Iteration	22
4.2.1	Iteration over an Iterable	23
4.2.2	Ranges and the Half-Open Convention	23
4.2.3	Condition-Controlled Iteration	25
4.2.4	Early Termination	26
5	References	27

1 Introduction

Programming provides a way to express computational processes in a form that a computer can execute. A computational problem begins with some input, applies a sequence of well-defined operations, and produces an output. The nature of the input and output depends on the

problem: they may consist of numbers, text, images, measurements, records, or more complex structures.

An **algorithm** describes the finite sequence of steps used to perform this transformation. It specifies what must be done without necessarily prescribing how those steps are expressed in a particular programming language. A **program** is a concrete representation of one or more algorithms, together with the data and supporting logic required for their execution.

The relationship can be represented schematically as:

Input $\longrightarrow$ Algorithm $\longrightarrow$ Output.

For example, consider the problem of finding the largest measurement in a collection. The input is a sequence of measurements, the algorithm examines the values while maintaining the largest value encountered so far, and the output is the maximum measurement. The same algorithmic idea can be implemented in different programming languages and executed on different machines.

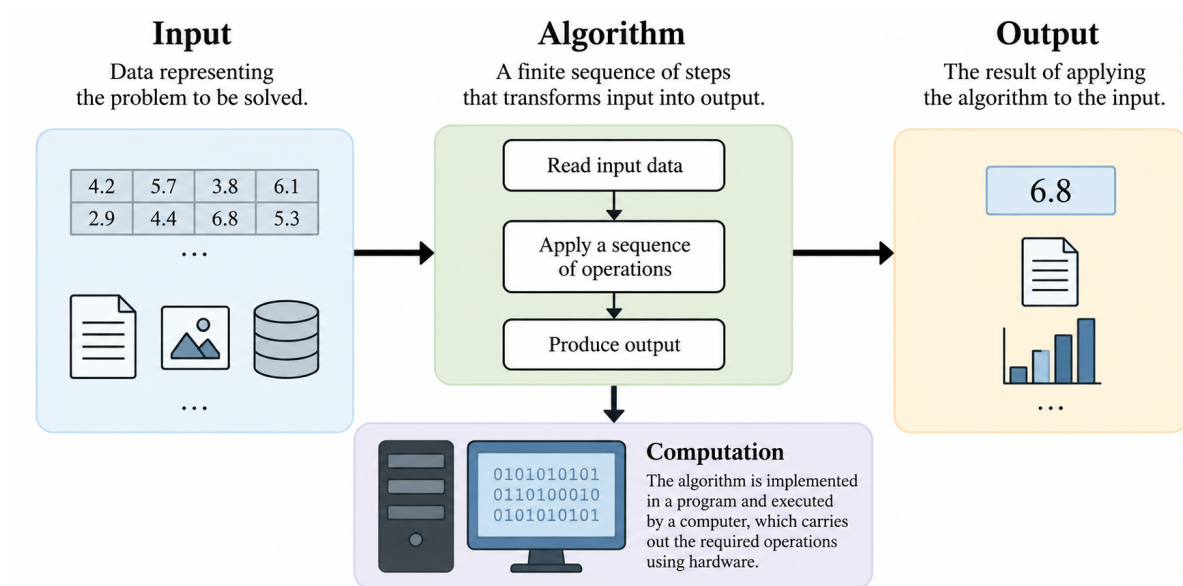


Figure 1: A computational process transforms input data into output through an algorithm implemented and executed by a computer.

Figure Figure 1 illustrates this general relationship. The algorithm provides the logical procedure, while computation refers to carrying out the operations described by that procedure. In practical computing systems, those operations are ultimately executed by hardware, although programming languages provide abstractions that allow programs to be expressed without working directly with machine instructions.

1.1 Python as a Programming Language

Python is a high-level, general-purpose programming language. It provides abstractions for representing data, expressing computations, organising programs, and building larger software systems. The language includes a relatively small core together with a large standard library and an extensive ecosystem of external packages.

Python programs are written as source code. A source file conventionally uses the `.py` extension and contains statements that can define objects, perform computations, control execution, define functions and classes, or interact with external resources.

For example:

```
1 measurements = [4.2, 5.7, 3.8, 6.1]
2 largest = max(measurements)
3
4 print(largest)
```

The source code describes the computation, but it is not executed directly by the processor in this form. Python implementations provide the machinery required to interpret the language and execute its instructions. This separation allows the programmer to work with concepts such as numbers, strings, functions, collections, and objects rather than processor instructions and memory addresses.

Python can be used interactively, where expressions and statements are evaluated as they are entered, or through scripts containing a sequence of instructions to be executed. Interactive execution is useful for exploration and experimentation, while scripts provide a persistent and reproducible representation of a computation. Both forms use the same language and follow the same underlying object model.

The usefulness of Python does not come only from its syntax. Much of its behaviour follows from a small number of underlying ideas: values are represented by objects, names refer to those objects, operations are defined by their types, and larger abstractions are constructed by combining objects and behaviours. Understanding these ideas is more important than memorising individual language constructs.

2 Objects, Types, and Names

A useful starting point for understanding Python is that computation operates on **objects**. Numbers, strings, lists, functions, classes, modules, and many other entities manipulated by a Python program are represented as objects.

Every object has three fundamental properties:

- an **identity**, which distinguishes it from other objects during its lifetime;
- a **type**, which determines the operations and behaviour supported by the object;
- a **value**, representing the information associated with the object.

Consider:

```
1 temperature = 21.5
```

It is tempting to describe this statement by saying that a variable called `temperature` contains the value 21.5. Although convenient in simple examples, that description becomes misleading once aliasing, mutability, function arguments, and container objects are considered.

A more accurate model is that Python creates or obtains an object representing the floating-point value 21.5 and binds the name `temperature` to that object:

`temperature` → float object representing 21.5.

The assignment statement therefore establishes a relationship between a **name** and an **object**.

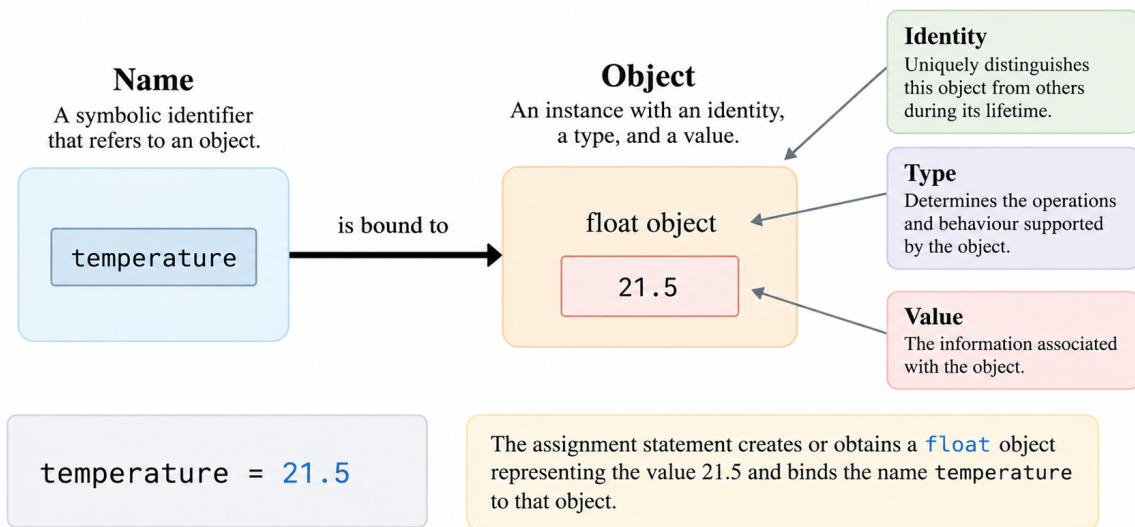


Figure 2: A Python name is bound to an object. The object has an identity, a type, and a value.

This distinction is fundamental because names and objects have different roles. A name belongs to a namespace and can later be bound to another object. The object exists independently of the particular name used to access it.

For example:

```
1 temperature = 21.5
2 temperature = 18.0
```

The second assignment does not modify the floating-point object representing 21.5. Instead, the name `temperature` is rebound to another object:

$$\begin{array}{c} \text{temperature} \longrightarrow 21.5 \\ \downarrow \\ \text{temperature} \longrightarrow 18.0. \end{array}$$

This behaviour is especially important because many Python objects are immutable: once created, their value cannot be changed. Reassigning a name that refers to an immutable object therefore changes the binding, not the object itself.

2.1 Dynamic Typing

The type belongs to the object rather than to the name. This is the basis of Python's **dynamic typing**.

A name can therefore refer to objects of different types at different points during execution:

```
1 result = 42
2 result = 3.14
3 result = "complete"
```

The name `result` has not changed its own type. Rather, it has successively been bound to an int, a float, and a str object.

The type of an object can be inspected with `type()`:

```
1 value = 42
2
3 type(value)
```

which reports `int`.

Types are important because they determine the operations that an object supports and the meaning of those operations. Addition, for example, has different behaviour depending on the operands:

```
1 2 + 3
2 "data" + "base"
```

The first expression performs numerical addition, while the second performs string concatenation. The + operator does not have a single meaning independent of its operands; its behaviour is determined by their types.

Python provides several fundamental built-in types. Among those used throughout these notes are:

Type	Purpose	Mutable
bool	Boolean values	No
int	Integer numbers	No
float	Floating-point numbers	No
complex	Complex numbers	No
str	Text	No
list	Mutable sequence	Yes
tuple	Immutable sequence	No
dict	Key-value mapping	Yes
set	Set of distinct objects	Yes
frozenset	Immutable set	No

The distinction between mutable and immutable objects will become important throughout the discussion of Python. It affects assignment, function calls, copying, hashing, and the behaviour of data structures.

2.2 Assignment and Aliasing

Because assignment binds names to objects, more than one name can refer to the same object. This situation is known as **aliasing**.

Consider:

```
1 alpha = [1, 2, 3]
2 beta = alpha
```

After the second assignment, no second list has been created. Both names refer to the same list:

alpha ↘
[1, 2, 3]
beta ↗

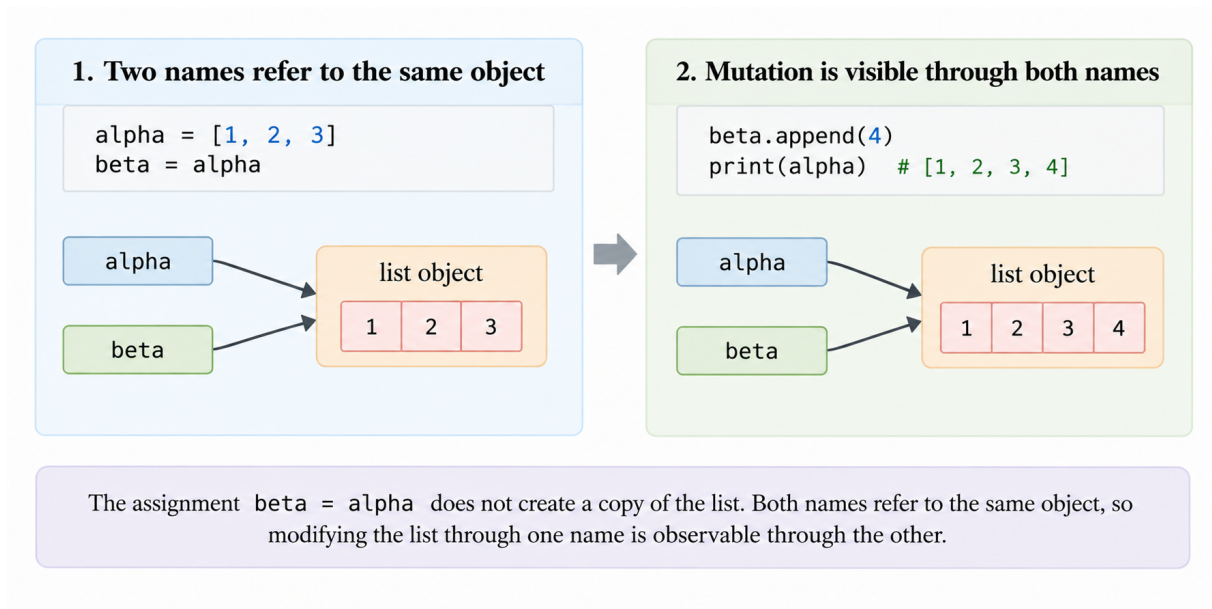


Figure 3: Two Python names may refer to the same object. Mutating that object is therefore observable through either name.

The consequence becomes visible when the object is modified:

```
1 beta.append(4)
2
3 print(alpha)
```

which produces:

```
1 [1, 2, 3, 4]
```

The assignment to `beta` did not copy the list. Since `alpha` and `beta` refer to the same mutable object, a mutation performed through one name is observable through the other.

This must be distinguished from **rebinding**:

```
1 beta = beta + [5]
```

The expression on the right creates a new list, after which beta is rebound to that new object. The original list remains associated with alpha.

The difference is therefore:

mutation changes an object	whereas	assignment changes a binding.
----------------------------	---------	-------------------------------

This distinction is one of the central ideas in Python's object model. It will reappear when considering mutable containers, function arguments, copying, class instances, and data structures.

2.3 Identity and Equality

Aliasing introduces an important distinction between two different questions that can be asked about objects:

1. Are two names referring to the same object?
2. Do two objects represent equivalent values?

Python provides different operators for these questions. The operators `is` and `is not` compare **identity**, while `==` and `!=` compare **equality**.

Consider two independently created lists:

```
1 alpha = [1, 2, 3]
2 beta = [1, 2, 3]
```

The lists contain the same values, so:

```
1 alpha == beta
```

evaluates to `True`. However:

```
1 alpha is beta
```

evaluates to `False`, because alpha and beta refer to two distinct list objects.

The distinction can be expressed as:

identity $\neq$ equality.

Identity concerns the object itself. Equality concerns whether two objects are considered equivalent according to the semantics of their types.

The situation changes if one name is assigned directly to the other:

```
1 alpha = [1, 2, 3]
2 beta = alpha
```

Now:

```
1 alpha is beta
```

evaluates to True, because both names refer to the same object. Since an object is also equal to itself under the usual equality semantics, `alpha == beta` is also True.

The two cases can therefore be distinguished conceptually as:

$$\begin{aligned} \text{alpha} == \text{beta} &\rightarrow \text{equivalent values,} \\ \text{alpha is beta} &\rightarrow \text{same object.} \end{aligned}$$

For most value comparisons, equality is the relevant relation. Identity testing is appropriate when the program specifically needs to determine whether two references designate the same object. A common example is comparison with None:

```
1 if result is None:
2     ...
```

Here the intention is not to ask whether result has a value equivalent to None, but whether it refers to the particular object used by Python to represent the absence of a value.

This distinction becomes even more important with user-defined classes. The meaning of equality can be defined as part of the behaviour of a type, whereas identity remains a property of the individual object. Equality will therefore be revisited when the Python object model and special methods are considered in greater detail.

2.4 Mutability and Immutability

The effect of aliasing depends strongly on whether the referenced object is **mutable** or **immutable**.

A mutable object can change its state after it has been created. Lists, dictionaries, and sets are common examples. An immutable object cannot change its value after creation. Numbers, strings, tuples, and frozensets are examples of immutable built-in types.

Consider a list:

```
1 values = [10, 20, 30]
```

The operation:

```
1 values.append(40)
```

changes the existing list object. The binding of the name `values` does not need to change:

$$\text{values} \longrightarrow [10, 20, 30]$$

becomes:

$$\text{values} \longrightarrow [10, 20, 30, 40].$$

The object has been mutated.

Now consider an integer:

```
1 count = 10
2 count = count + 1
```

An integer object cannot be changed from 10 into 11. The expression on the right-hand side is evaluated first, producing an integer object representing 11. The name `count` is then rebound to that object.

Conceptually:

$$\text{count} \longrightarrow 10$$

is replaced by:

$$\text{count} \longrightarrow 11.$$

The integer object representing 10 has not been modified.

This distinction explains why superficially similar operations may have different consequences when aliases exist.

For example:

```
1 alpha = [1, 2]
2 beta = alpha
3
4 alpha.append(3)
```

Since the list is mutable, both names still refer to the modified object:

```
1 alpha -> [1, 2, 3]
2 beta -> [1, 2, 3]
```

By contrast:

```
1 alpha = 10
2 beta = alpha
3
4 alpha = alpha + 1
```

does not alter the object to which beta refers. Instead, alpha is rebound:

```
1 alpha -> 11
2 beta -> 10
```

Mutability is therefore not simply a property used to classify built-in types. It has direct consequences for how state changes propagate through a program.

An important subtlety is that the immutability of a container does not imply that every object referenced by the container is itself immutable. A tuple, for example, cannot have its references replaced after creation, but it may contain a mutable object:

```
1 record = ("sensor-01", [21.4, 21.7])
2 record[1].append(22.0)
```

The tuple itself has not been modified: it still refers to the same string and the same list. The list contained within it, however, is mutable and has changed.

Thus, immutability concerns the state of the object under consideration, not necessarily the state of every object reachable from it.

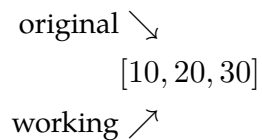
2.5 Copying Objects

Aliasing is useful when several parts of a program are intended to operate on the same object. In other situations, independent state is required.

Suppose an existing collection is used as the starting point for another collection:

```
1 original = [10, 20, 30]
2 working = original
```

This does not create an independent list. The assignment creates another reference to the same object:



Consequently:

```
1 working.append(40)
```

also changes what is observed through original.

If an independent outer list is required, a new list can be constructed:

```
1 working = list(original)
```

or, equivalently for a list:

```
1 working = original.copy()
```

Now original and working refer to distinct list objects. Adding or removing an element from one list does not change the other.

For flat collections containing immutable elements, this distinction is often sufficient. Nested mutable structures reveal a deeper issue.

Consider:

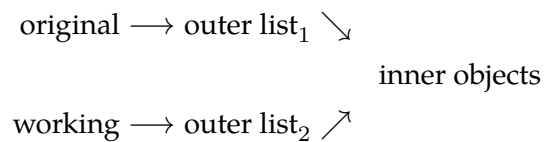
```

1 original = [
2     ["A", 10],
3     ["B", 20],
4 ]
5
6 working = original.copy()

```

Two list objects now exist at the outer level, but their elements still refer to the same inner lists.

The structure is conceptually:



This is a **shallow copy**. The outer container has been duplicated, but the objects referenced by that container have not.

Therefore:

```

1 working[0][1] = 99

```

also changes:

```

1 original[0]

```

because:

```

1 working[0] is original[0]

```

is True.

When the nested objects must also be independent, a **deep copy** may be required. Python provides this operation through the copy module:

```

1 import copy
2
3 working = copy.deepcopy(original)

```

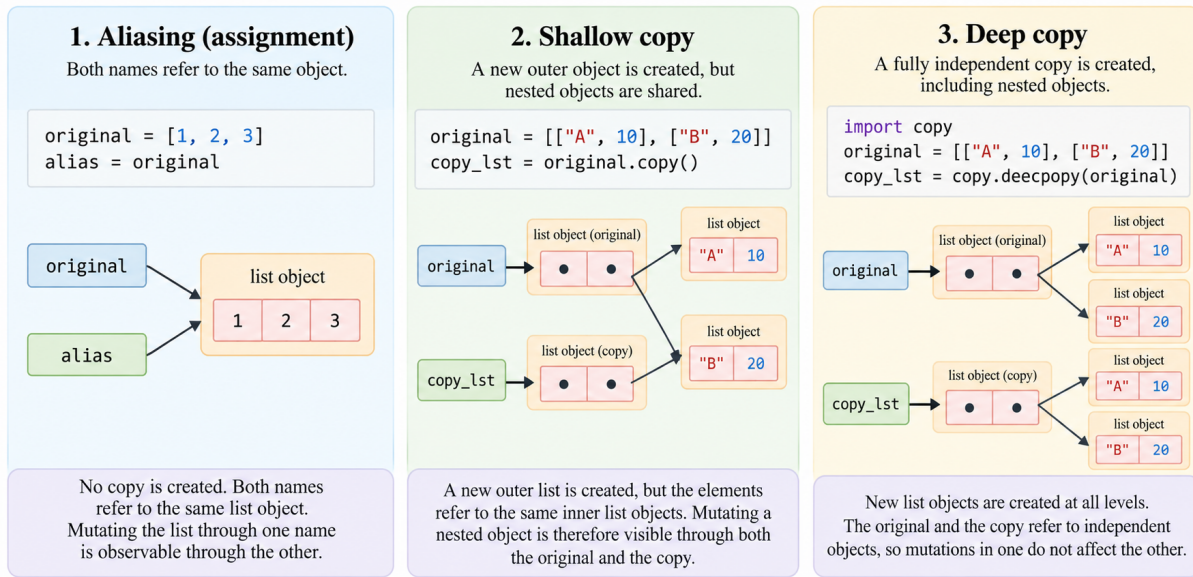


Figure 4: Aliasing, shallow copying, and deep copying differ in which objects are shared between the original structure and its copy.

A deep copy recursively constructs independent copies of the objects contained within the structure rather than merely duplicating the outer container.

The three situations can be summarised as:

assignment → same object,
shallow copy → new outer object, shared nested objects,
deep copy → independent copied structure.

Deep copying should not be understood as a universally preferable form of copying. Sharing objects can be intentional and efficient, while duplicating an entire object structure may be unnecessary or inappropriate. The relevant question is whether the state represented by the objects is intended to be shared.

The distinction between references and copies will become particularly important when working with nested containers and data structures. Many Python containers are **referential structures**: they store references to objects rather than embedding independent copies of those objects. Understanding what is copied and what remains shared is therefore necessary for reasoning correctly about their behaviour.

3 Expressions and Operators

Objects become useful in a computation through the operations that can be performed on them. An **expression** combines objects, names, literals, operators, and function calls to produce a value.

For example:

```
1 subtotal = price * quantity
```

contains the expression:

```
1 price * quantity
```

which is evaluated before the resulting object is bound to the name subtotal.

The meaning of an operator depends on the types of its operands. The expression:

```
1 2 + 3
```

performs numerical addition, while:

```
1 "data" + "science"
```

performs string concatenation.

This is not merely syntactic convenience. Operators are part of the behaviour provided by types. Python uses a common notation for operations that may have different meanings for different kinds of objects.

Expressions may combine several operations:

```
1 result = a + b * c
```

In such cases, Python applies a defined operator precedence. Multiplication is evaluated before addition, so the expression is interpreted as:

```
1 result = a + (b * c)
```

rather than:

```
1 result = (a + b) * c
```

Explicit parentheses should be used when they clarify the intended grouping, even when the default precedence would produce the same result.

Operators can broadly express arithmetic, comparison, identity, membership, and logical relationships. Their detailed behaviour will be introduced together with the types and control structures for which they are relevant rather than treated as an isolated catalogue of syntax.

This is particularly useful for Boolean expressions. Conditions such as:

```
1 temperature > 30
```

or:

```
1 value is None
```

do not merely perform computations; they produce Boolean results that can determine which parts of a program are executed. This provides the connection between expressions and **control flow**, which determines how execution proceeds through a program.

3.1 Boolean Expressions and Logical Operators

Many computations require decisions rather than only the production of numerical or textual values. A program may need to determine whether a measurement exceeds a threshold, whether a collection contains an element, or whether several conditions hold simultaneously.

These decisions are expressed through Boolean values. Python provides the type `bool`, whose two possible values are:

```
1 True
2 False
```

Comparison operators produce Boolean results. For example:

```
1 temperature = 28.5
2
3 temperature > 25
```

evaluates to `True`, while:

```
1 temperature < 20
```

evaluates to False.

The usual ordering comparisons are:

Operator	Meaning
$a < b$	a is less than b
$a \leq b$	a is less than or equal to b
$a > b$	a is greater than b
$a \geq b$	a is greater than or equal to b
$a == b$	a and b are equal
$a != b$	a and b are not equal

As discussed previously, `is` and `is not` answer a different question: they compare object identity rather than value equality.

Python also supports chained comparisons. A mathematical condition such as:

$$0 \leq x < 1$$

can be written directly as:

```
1 0 <= x < 1
```

This expresses the intended relationship more naturally than writing the two comparisons separately.

More complex conditions can be constructed with the logical operators `not`, `and`, and `or`.

For propositions P and Q , their behaviour can be summarised by the following truth table:

P	Q	P and Q	P or Q
False	False	False	False
False	True	False	True
True	False	False	True
True	True	True	True

Negation reverses the truth value:

P	not P
False	True
True	False

These operators allow conditions to represent the actual logical requirements of a problem. For example:

```
1 temperature > 20 and temperature < 30
```

requires both comparisons to hold, while:

```
1 status == "warning" or status == "critical"
```

requires at least one of them.

Logical expressions should generally be written to make the underlying condition evident. Parentheses can be useful when several operators are combined, even when Python's precedence rules would make them unnecessary:

```
1 (is_valid and has_data) or use_default
```

The purpose of such expressions is not merely to produce True or False. They provide the logical basis for controlling which computations a program performs.

3.1.1 Truth Values

Conditions in Python do not always require an explicit bool object. Objects of other types can be evaluated in a Boolean context.

Several common values are considered false, including numeric zero, None, and empty containers:

```
1 0
2 0.0
3 ""
4 []
5 {}
6 set()
7 None
```

Nonzero numbers and nonempty containers are generally considered true.

This allows conditions such as:

```
1 records = [...]
2
3 if records:
4     ...
```

rather than:

```
1 if len(records) > 0:
2     ...
```

The first form expresses the relevant question directly: whether records contains data.

Truth-value testing should nevertheless be used when its meaning is clear. Different states may need to be distinguished explicitly. For example, an empty collection and None may both evaluate as false while representing different situations in a program.

3.1.2 Short-Circuit Evaluation

The operators and and or are evaluated from left to right and use **short-circuit evaluation**. Python evaluates the second operand only when it is necessary to determine the result.

For:

```
1 P and Q
```

if P is false, the complete expression cannot be true, regardless of Q. Therefore Q does not need to be evaluated.

Similarly, for:

```
1 P or Q
```

if P is true, the complete expression is already true and Q does not need to be evaluated.

This behaviour has practical consequences. Consider a search through a sequence:

```
1 index = 0
2
3 while index < len(data) and data[index] != target:
4     index += 1
```

The first condition verifies that `index` is valid before the expression attempts to access:

```
1 data[index]
```

When `index == len(data)`, the first condition is false. Because `and` short-circuits, the second condition is not evaluated, preventing an invalid access beyond the end of the sequence.

The order of the conditions is therefore significant. Writing:

```
1 while data[index] != target and index < len(data):  
2     ...
```

would not provide the same protection, because `data[index]` would be evaluated first.

Short-circuit evaluation is consequently more than an optimisation. It can be part of the correctness of an expression.

4 Control Flow

A program consisting only of expressions evaluated sequentially would follow the same path every time it executes. Most computations require greater flexibility: some operations should occur only under certain conditions, while others must be repeated until a task is complete.

Control flow determines the order in which statements are executed.

Python provides two fundamental mechanisms for changing sequential execution:

- **selection**, which chooses among alternative paths;
- **iteration**, which repeats a computation.

These mechanisms are expressed primarily through conditional statements and loops.

Python uses indentation to define blocks of statements. Indentation is therefore part of the structure of the language rather than merely a formatting convention.

For example:

```
1 if temperature > 30:  
2     alert = True  
3     message = "High temperature"  
4  
5 log_temperature(temperature)
```

The two indented statements belong to the conditional block. The final statement lies outside it and is executed regardless of the condition.

4.1 Conditional Execution

An if statement allows execution to depend on a condition evaluated at runtime.

Consider a simple classification:

```
1 if temperature > 30:
2     status = "high"
3 else:
4     status = "normal"
```

Exactly one of the two assignments is performed. The selected path depends on the Boolean value of:

```
1 temperature > 30
```

More than two mutually exclusive cases can be represented with elif:

```
1 if temperature < 10:
2     status = "low"
3 elif temperature <= 30:
4     status = "normal"
5 else:
6     status = "high"
```

The conditions are evaluated in order. Once one evaluates to true, its corresponding block is executed and the remaining alternatives are skipped.

This ordering matters whenever conditions overlap. Consider:

```
1 if score >= 50:
2     grade = "pass"
3 elif score >= 80:
4     grade = "distinction"
```

A score of 90 satisfies the first condition, so the second condition is never reached. The intended classification requires the more restrictive condition first:

```
1 if score >= 80:
2     grade = "distinction"
3 elif score >= 50:
4     grade = "pass"
5 else:
6     grade = "fail"
```

Conditional logic therefore involves more than writing syntactically valid comparisons. The branches must partition the relevant cases in a way that represents the problem correctly.

Several independent if statements should also be distinguished from a single if–elif–else chain. With independent statements:

```
1 if condition_a:
2     ...
3
4 if condition_b:
5     ...
```

both blocks may execute. In an if–elif chain, at most one branch is selected.

This difference reflects two different logical structures: independent conditions versus mutually exclusive alternatives.

4.2 Iteration

Many computational tasks involve applying an operation repeatedly. A collection may need to be processed element by element, a search may continue until a target is found, or a numerical method may iterate until a convergence criterion is satisfied.

Python provides two principal loop constructs:

```
1 for
2 while
```

They both express repetition, but they correspond to different ways of describing when repetition should continue.

4.2.1 Iteration over an Iterable

A for loop consumes values from an **iterable**:

```
1 measurements = [4.2, 5.7, 3.8, 6.1]
2
3 for value in measurements:
4     print(value)
```

The important idea is that Python's for loop is fundamentally an iteration over values, not a counter-based loop. During each iteration, the loop variable is bound to the next object produced by the iterable.

For example, a maximum can be computed as:

```
1 measurements = [4.2, 5.7, 3.8, 6.1]
2
3 largest = measurements[0]
4
5 for value in measurements[1:]:
6     if value > largest:
7         largest = value
```

The loop directly expresses the operation being performed: examine each remaining measurement and update the current maximum when a larger value is encountered.

This pattern appears throughout algorithms:

initial state → process next element → update state → repeat.

The variable `largest` represents the state accumulated by the computation. After each iteration, it contains the largest value encountered so far.

4.2.2 Ranges and the Half-Open Convention

Some computations require positions rather than only values. Python provides `range` for generating sequences of integers suitable for iteration.

The expression:

```
1 range(n)
```

represents the integers:

$$0, 1, 2, \dots, n - 1.$$

The upper bound n is excluded. More generally:

```
1 range(start, stop)
```

follows the half-open interval:

$$[\text{start}, \text{stop}).$$

This convention is particularly useful for indexing. A sequence of length n has valid positions:

$$0, 1, \dots, n - 1,$$

which correspond exactly to:

```
1 range(n)
```

For example:

```
1 for index in range(len(data)):
2     print(index, data[index])
```

There is, however, no reason to introduce an index when only the values are needed. If the position and value are both relevant, Python provides `enumerate`:

```
1 for index, value in enumerate(data):
2     print(index, value)
```

This expresses the intention more directly than manually managing an index.

4.2.3 Condition-Controlled Iteration

A while loop is appropriate when repetition is governed by a condition rather than by consuming a predefined iterable:

```
1 while condition:
2     ...
```

The condition is evaluated before every iteration. Execution continues while it remains true.

Consider an iterative approximation in which successive estimates are computed until the change becomes sufficiently small:

```
1 estimate = initial_estimate
2
3 while error > tolerance:
4     estimate = update(estimate)
5     error = compute_error(estimate)
```

The number of iterations is not necessarily known beforehand. Termination depends on the evolving state of the computation.

This distinguishes the two principal loop forms conceptually:

for → iterate over values from an iterable,
while → repeat while a condition holds.

A while loop must eventually make progress toward making its condition false unless nontermination is intentional. Failure to update the relevant state produces an infinite loop.

For example:

```
1 index = 0
2
3 while index < len(data):
4     process(data[index])
5     index += 1
```

terminates because index increases on each iteration and eventually reaches len(data).

This idea of **progress toward termination** becomes important when reasoning about algorithm correctness. A loop should not only perform the intended operation during each iteration; there must also be a reason why the process eventually stops.

4.2.4 Early Termination

Sometimes a loop does not need to examine every possible element. Once the required information has been obtained, continuing would perform unnecessary work.

A search is a typical example:

```
1 found = False
2
3 for value in data:
4     if value == target:
5         found = True
6         break
```

The `break` statement terminates the nearest enclosing loop immediately.

The `continue` statement has a different purpose. It terminates only the current iteration and proceeds with the next one:

```
1 for value in data:
2     if value < 0:
3         continue
4
5     process(value)
```

Here negative values are skipped while iteration continues over the remaining elements.

Both constructs can simplify control flow when used deliberately. Excessive use, however, can make the execution path difficult to follow. A loop should generally retain a clear termination condition and a clear purpose.

Python also allows a loop to have an `else` clause. The `else` block executes when the loop finishes normally rather than through `break`.

This is useful for searches:

```
1 for value in data:
2     if value == target:
3         print("Target found")
4         break
5 else:
6     print("Target not found")
```

The else clause expresses a precise condition: the loop exhausted its iterable without encountering a break.

This behaviour can initially appear unusual, but it avoids maintaining a separate Boolean flag when the relevant distinction is simply whether the search terminated early.

Control structures provide the machinery for expressing algorithmic processes, but larger programs also require a way to isolate computations behind well-defined interfaces. This is the role of **functions**, which allow behaviour to be named, parameterised, reused, and composed.

5 References

- [1] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data structures and algorithms in python*. Wiley Hoboken, NJ, USA, 2013.